

In Python programming language, operators help you to work with and manipulate data effectively. There are seven different categories of operators in Python, and you will learn about each one in this article with the help of examples.

What are Operators in Python?

OPERATORS IN PYTHON

Types	Operators
Arithmetic Operators	+, -, *, /, %, **, //
Assignment Operators	+=, -=, *=, /=, %=, **=, //=
Comparison/Relational Operators	==, !=, >, <, >=, <=
Logical Operators	AND, OR, NOT
Bitwise Operators	&, , ^, ~, <<, >>
Identity Operators	is, is not
Membership Operators	in, not in

@codewithpj

Photo by Author

Python operators are symbols or keywords used to perform arithmetic and logical operations on values and variables of any data type. The value that the operator is applied on is called the **operand**.

Types of Python Operators

There are seven operators in Python:

1. Arithmetic Operators
2. Assignment Operators
3. Comparison Operators
4. Logical Operators
5. Bitwise Operators
6. Identity Operators
7. Membership Operators

Arithmetic Operators

These operators are used to perform mathematical operations on variables. You can use these operators to do multiplication, addition, subtraction, division and much more. Below are some common arithmetic operators in Python:

- **Addition (+):** Used to add two operands.
- **Subtraction (-):** Used to subtract the right operand from the left operand.
- **Multiplication (*):** Used to multiply two operands.
- **Division (/):** Used to divide the left operand by the right operand.
- **Modulus (%):** Used to get the remainder of the division of the left operand by the right operand.
- **Exponentiation (**):** Used to raise the left operand to the power of the right operand.

- **Floor Division (//):** Used to perform division where the result is the quotient in which the digits after the decimal point are removed.

```
x = 5
y = 6
z = 7
print(x + y + z) #output is 18

print(x - y - z) #output is -8

print(x * y * z) #output is 210

print(x/y) #output is 0.8333333333333334

print(x%y) #output is 5

print(x**y) #output is 15625
print(x//y) #output is 0
```

Assignment Operators

Assignment operators are used to assign values to variables. **This operator assigns the value of the expression's right side to the operand on the left side.** There are eight types of assignment operators in Python:

- **Assignment operator(=):** used to assign values from the right side of the operator to the variable on the left side.
- **Addition assignment operator(+=):** used to add the right operand to the left operand and assign the result to the left operand.
- **Subtraction assignment operator (-=):** used to subtract the right operand from the left operand and assign the result to the left operand.
- **Multiplication assignment operator(*=):** used to multiply the right operand with the left operand and assign the result to the left operand.

- **Division assignment operator (/=):** used to divide the left operand by the right operand and assign the result to the left operand.
- **Modulus assignment operator (%=):** used to take the modulus of the two operands and assign the result to the left operand.
- **Exponentiation assignment operator (**=):** used to raise the left operand to the power of the right operand and assign the result to the left operand.
- **Floor division assignment operator (//=):** used to perform floor division on the operands and assign the result to the left operand.

```
a = 15
b = 5
a += b
print(a) #output is 20

a -= b
print(a) #output is 10

a *= b
print(a) #output is 75

a /= b
print(a) #output is 3

a %= b
print(a) #output is 0

a **= b
print(a) #output is 759375

a //= b
print(a) #output is 3
```

Comparison Operators

Also known as relational operators, they are used to compare different values or variables. These operators only return **True** or **False** according to the condition. There are six types of comparison operators namely:

- **Equality or Equal to operator(==):** used to compare if the values of two operands are equal.
- **Inequality or Not Equal to operator (!=):** used to compare if the values of two operands are not equal.
- **Greater than sign(>):** used to check if the left operand is greater than the right operand.
- **Less than sign(<):** used to check if the left operand is less than the right operand.
- **Greater than or Equal to sign(>=):** used to compare if the left operand is greater than or equal to the right operand.
- **Less than or Equal to sign(<=):** used to compare if the left operand is less than or equal to the right operand

```
a = 5
b = 4
print(a==b) #output is False
print(a!=b) #output is True
print(a>b) #output is True
print(a<b) #output is False
print(a>=b) #output is True
print(a<=b) #output is False
```

Logical Operators

In Python, logical operators are used to combine conditional statements. There are three logical operators in Python:

- **AND operator:** returns True if both statements are true.
- **NOT operator:** returns True if one of the statements is true.
- **OR operator:** reverses or negates the result.

```
a = 5
b = 4
print(a<5 and b<4) #output is False
print(a>=5 or b>4) #output is True
print(not(a<5 and b<4) #output is True
```

Truth Table for Python Logical Operators

TRUTH TABLE For Python Logical Operators

a	b	a and b	a or b	not (a)	not (b)
T	T	T	T	F	F
T	F	F	T	F	T
F	T	F	T	T	F
F	F	F	F	T	T

@codewithpj

Photo by Author

Bitwise Operators

In high school, you were taught about binary numbers. Remember, the Ones(1s) and Zeros(0s). In Python, bitwise operators help you to work with and manipulate these binary numbers in the form of bits of data. In essence, these operators perform bit-by-bit operations on specified values. There are six Python bitwise operators:

- **AND (&) Operator:** If both bits are 1, the result is 1; otherwise, it's 0.

```
a = 5
b = 4
print(a&b) #output is 4
```

Remember, in binary, 5 = 0101 and 4 = 0100. Following the principle of the AND operator, the result will be 0100 which is 4.

- **OR (|) Operator:** If at least one of the bits is 1, the result is 1; otherwise, it's 0.

```
a = 5
b = 4
print(a|b)
#output is 5
```

- **XOR (^) Operator:** If the bits are different, the result is 1; if they are the same, the result is 0.

```
a = 5
b = 4
print(a^b)
#output is 1
```

- **NOT (~) Operator:** This operator inverts the bits of its operand. So, it changes 1 to 0 and 0 to 1. The NOT operator is a **unary operator** meaning it only performs operation on the value of a single variable as shown below:

```
a = 5
b = 4
print(~b)
#output is -5
```

Writing 'print(a~b)' will return a SyntaxError.

- **Left Shift (<<) operator:** The left shift operator moves the binary digits of a number to the left by a certain number of positions specified, filling it with zeros from the right.

For example, if you have 5, which is 101 in binary, and you use the left shift operator to shift it by 2 positions, you have:

```
print(5 << 2)
#output is 20
```

The result will be 20 which is 10100 in binary. This means that 101 shifted 2 positions and added two zeros from the right.

- **Right Shift (>>) Operator:** The right shift operator moves the binary digits of a number to the right by a certain number of positions specified, filling it with zeros from the left.

So, if you have 20, which is 10100 in binary, and you use the right shift operator (>>) to shift it by 2 positions, you have:

```
print(20 >> 2)
#output is 5
```

The result will be 5 which is 101 in binary. This means that 20 shifted 2 positions, adding two “invisible zeros” from the left.

Identity Operators

Identity operators are used to compare the memory location of two objects. The output from using an identity operator is always a Boolean value, that means, it's either True or False.

There are two types of identity operators in Python: **is** and **is not**. ‘is’ confirms if the two objects have the same memory location, while “is not” determines if two objects do not have the same memory location.

Note that identity operators are not the same as equality operators (==). An equality operator compares the objects, not their memory location.

```
a = 5
b = 5
print(a is b)
#output is True
print (a is not b)
#output is False
```

To check the memory location of a value, you can use the `id()` method as shown below:

```
a = 5
b = 5
print(id(a)) #output is 499361802696
print(id(b)) #output is 499361802696
```

Membership Operators

In Python, membership operators are used to check if a character, substring, value, or variable is present in a sequence. This sequence could be a string, tuple, list, or a set.

There are two types of membership operators, namely: **in** and **not in**. The 'in' operator will return True if the specified character is present in that sequence. Otherwise, it will return False.

The "not in" operator will return True if the specified character is not present in that sequence. Otherwise, it will return False.

```
name = 'Praise'
print('P' in 'Praise') #output is True
```